

Práctica 08. JDBC & Psycopg.

Gallardo Valdez Brayan Alexis Granados Rovira José Pablo
Lozano León Diego Márquez Corona Danna Lizette
Paredes González Emiliano Sebastián Tapia Anrubio Irving Axel

03 Mayo 2026

1. Introducción

En esta práctica se realizó una aplicación web conectada a una base de datos en PostgreSQL, tomando como base el proyecto “Una farmacia de otro mundo”. El objetivo principal fue trabajar con operaciones CRUD, es decir, poder insertar, consultar, actualizar y eliminar información dentro de algunas tablas de la base de datos. Esto permite manejar los datos del sistema de una forma más ordenada y útil.

Para conectar la aplicación con la base de datos se utilizó una tecnología de comunicación, como JDBC en el caso de Java con Spring Boot, o Psycopg2 en el caso de Python con Django REST. La práctica pedía trabajar con al menos dos tablas de la base de datos y aplicar en cada una de ellas las operaciones básicas de registro, consulta, modificación y eliminación de datos.

También se realizaron pruebas con Postman, una herramienta que permite comprobar si los endpoints de la aplicación REST funcionan correctamente. Gracias a estas pruebas, se pudo verificar que la aplicación sí se comunicaba con la base de datos y que las consultas se ejecutaban de manera adecuada para administrar la información del sistema.

Se usó Python, Django, conexión con Postman, uso de DBeaver, PostgreSQL

2. Tablas utilizadas

Para esta práctica se eligieron dos tablas de la base de datos: **CLIENTE** y **SUCURSAL**. Estas tablas aparecen definidas directamente en el archivo `DDL.sql` del proyecto y también son las tablas sobre las que se implementaron las operaciones CRUD dentro de la aplicación REST. La elección cumple con lo solicitado en la práctica, ya que se pide trabajar con al menos dos tablas de la base de datos e implementar sobre ellas operaciones para insertar, eliminar, actualizar, consultar todos los registros y consultar un registro por identificador.

- **Tabla CLIENTE**

La tabla **CLIENTE** se utiliza para almacenar la información de los clientes registrados en la farmacia. En el archivo `DDL.sql`, esta tabla contiene datos personales, domicilio, método de pago y campos relacionados con el acceso en línea del cliente.

Los atributos definidos para esta tabla en el proyecto son:

```
1 CLIENTE(  
2     IDCliente ,  
3     Nombre ,  
4     ApellidoPaterno ,  
5     ApellidoMaterno ,  
6     FechaNacimiento ,  
7     Calle ,  
8     NumExterior ,  
9     NumInterior ,  
10    Colonia ,  
11    Estado ,  
12    MetodoPago ,  
13    NumeroTarjeta ,  
14    VencimientoTarjeta ,  
15    Usuario ,  
16    Contraseña ,  
17    EsClienteEnLinea ,  
18    EsClienteFisico ,  
19    EsPaciente  
20 )
```

Escogimos esta tabla porque representa a una de las entidades principales del sistema. Los clientes pueden realizar compras, asistir a consultas, recibir recetas y generar tickets dentro de la farmacia. Además, en la aplicación REST existe una carpeta específica llamada cliente, donde se implementan las consultas necesarias para crear, consultar, actualizar y eliminar registros de esta tabla. En el archivo `cliente/queries.py` se manejan las siguientes operaciones:

- `get_all_clientes()`
- `get_cliente_by_id(id_cliente)`
- `create_cliente(data)`
- `update_cliente(id_cliente, data)`
- `delete_cliente(id_cliente)`

Así, CLIENTE fue una de las tablas usadas directamente para probar el funcionamiento de la conexión entre Django REST, Psycopg2 y PostgreSQL.

■ Tabla SUCURSAL

La tabla **SUCURSAL** se utiliza para guardar la información de las sucursales físicas de la farmacia. En el archivo `DDL.sql`, esta tabla contiene el identificador de la sucursal, su nombre y los datos principales de su dirección.

Los atributos definidos para esta tabla son:

```
1 SUCURSAL (
2     IDSucursal ,
3     Nombre ,
4     Calle ,
5     NumExterior ,
6     NumInterior ,
7     Colonia ,
8     Estado
9 )
```

Esta tabla también fue seleccionada porque representa una parte central del sistema. Las **sucursales** son los lugares donde se realizan ventas, consultas médicas y operaciones relacionadas con medicamentos, insumos, clientes y personal.

En la aplicación REST existe una carpeta específica llamada `sucursal`, donde se implementan las operaciones CRUD para esta tabla. En el archivo `sucursal/queries.py` se manejan las siguientes funciones:

- `get_all_sucursales()`
- `get_sucursal_by_id(id_sucursal)`
- `create_sucursal(data)`
- `update_sucursal(id_sucursal, data)`
- `delete_sucursal(id_sucursal)`

Estas funciones permiten consultar todas las sucursales, buscar una sucursal por su identificador, insertar una nueva sucursal, actualizar sus datos y eliminarla de la base de datos.

En conjunto, las tablas **CLIENTE** y **SUCURSAL** ayudan a mostrar cómo funcionan las operaciones CRUD dentro de la aplicación REST. La tabla **CLIENTE** guarda información de los usuarios o pacientes que usan los servicios de la farmacia, mientras que **SUCURSAL** contiene los datos de los establecimientos donde se ofrecen dichos servicios.

Por esta razón, ambas tablas son una buena opción para probar la conexión entre la base de datos y la API desarrollada con Django REST y Psycopg2, ya que permiten comprobar acciones básicas como registrar, consultar, modificar y eliminar información.

3. Operaciones CRUD y queries utilizadas

Se implementaron operaciones CRUD para las tablas **CLIENTE** y **SUCURSAL**, usando Django REST Framework para recibir las peticiones HTTP y Psycopg2 para ejecutar directamente las consultas SQL en PostgreSQL. Se realizó para al menos dos tablas, se implementara insertar, eliminar, actualizar, consultar todos los registros y consultar un registro por identificador.

3.1. CRUD de la tabla CLIENTE

3.1.1. Create, Insertar cliente

Esta operación permite registrar un nuevo cliente en la base de datos. Recibe los datos enviados desde la aplicación REST y los inserta en la tabla **CLIENTE**.

```
1 INSERT INTO CLIENTE (  
2     IDCliente, Nombre, ApellidoPaterno, ApellidoMaterno,  
3     FechaNacimiento, Calle, NumExterior, NumInterior,  
4     Colonia, Estado, MetodoPago, NumeroTarjeta,  
5     VencimientoTarjeta, Usuario, Contraseña,  
6     EsClienteEnLinea, EsClienteFisico, Espaciente  
7 ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s,  
8           %s, %s, %s, %s, %s, %s, %s, %s, %s, %s);
```

Se usa **INSERT INTO** para crear un nuevo registro. Los **%s** son marcadores usados en Psycopg2 para insertar los valores de forma segura.

3.1.2. Read, Consultar todos los clientes

Esta operación permite buscar un cliente específico usando su identificador **IDCliente**.

```
1 SELECT IDCliente, Nombre, ApellidoPaterno, ApellidoMaterno,  
2     FechaNacimiento, Calle, NumExterior, NumInterior,  
3     Colonia, Estado, MetodoPago, NumeroTarjeta,  
4     VencimientoTarjeta, Usuario, Contraseña,  
5     EsClienteEnLinea, EsClienteFisico, Espaciente  
6 FROM CLIENTE  
7 WHERE IDCliente = %s;
```

Aquí se usamos **WHERE IDCliente = %s** para filtrar únicamente el cliente cuyo identificador coincida con el valor recibido en la URL.

3.1.3. Update, Actualizar cliente

Esta operación modifica los datos de un cliente existente. El registro se localiza mediante IDCliente

```
1 UPDATE CLIENTE
2 SET Nombre           = %s,
3     ApellidoPaterno  = %s,
4     ApellidoMaterno  = %s,
5     FechaNacimiento  = %s,
6     Calle             = %s,
7     NumExterior       = %s,
8     NumInterior       = %s,
9     Colonia           = %s,
10    Estado            = %s,
11    MetodoPago        = %s,
12    NumeroTarjeta     = %s,
13    VencimientoTarjeta = %s,
14    Usuario           = %s,
15    Contraseña        = %s,
16    EsClienteEnLinea  = %s,
17    EsClienteFisico   = %s,
18    EsPaciente        = %s
19 WHERE IDCliente = %s;
```

Esta query usa UPDATE para modificar los campos del cliente y WHERE para asegurarse de que solo se actualice el registro indicado.

3.1.4. Delete, Eliminar cliente

Esta operación elimina un cliente de la tabla usando su id.

```
1 DELETE FROM CLIENTE
2 WHERE IDCliente = %s;
```

Se usa DELETE FROM para borrar el registro y WHERE para evitar eliminar todos los clientes por accidente.

3.2. CRUD de la tabla SUCURSAL

3.2.1. Create, Insertar sucursal

Esta operación permite agregar una nueva sucursal a la base de datos.

```
1 INSERT INTO sucursal (
2     idsucursal, nombre, calle, numexterior,
3     numinterior, colonia, estado
4 ) VALUES (%s, %s, %s, %s, %s, %s, %s);
```

La consulta usa INSERT INTO para crear un nuevo registro en la tabla sucursal.

3.2.2. Read, Consultar todas las sucursales y Consultar sucursal por ID

- Consultar todas las sucursales: Esta operación obtiene todas las sucursales registradas en la base de datos.

```
1 SELECT idsucursal, nombre, calle, numexterior,  
   numinterior, colonia, estado  
2 FROM sucursal  
3 ORDER BY idsucursal;
```

Se usa `SELECT` para recuperar los datos principales de cada sucursal y `ORDER BY` para ordenarlas por su identificador.

- Consultar sucursal por ID: Esta operación busca una sucursal específica mediante `idsucursal`.

```
1 SELECT *  
2 FROM sucursal  
3 WHERE idsucursal = %s;
```

La condición `WHERE idsucursal = %s` permite obtener únicamente la sucursal solicitada.

3.2.3. Update, Actualizar sucursal

Esta operación modifica los datos de una sucursal existente.

```
1 UPDATE sucursal  
2 SET nombre = %s,  
3     calle = %s,  
4     numexterior = %s,  
5     numinterior = %s,  
6     colonia = %s,  
7     estado = %s  
8 WHERE idsucursal = %s;
```

Se usa `UPDATE` para cambiar los datos de la sucursal y `WHERE` para identificar cuál registro debe modificarse.

3.2.4. Delete, Eliminar sucursal

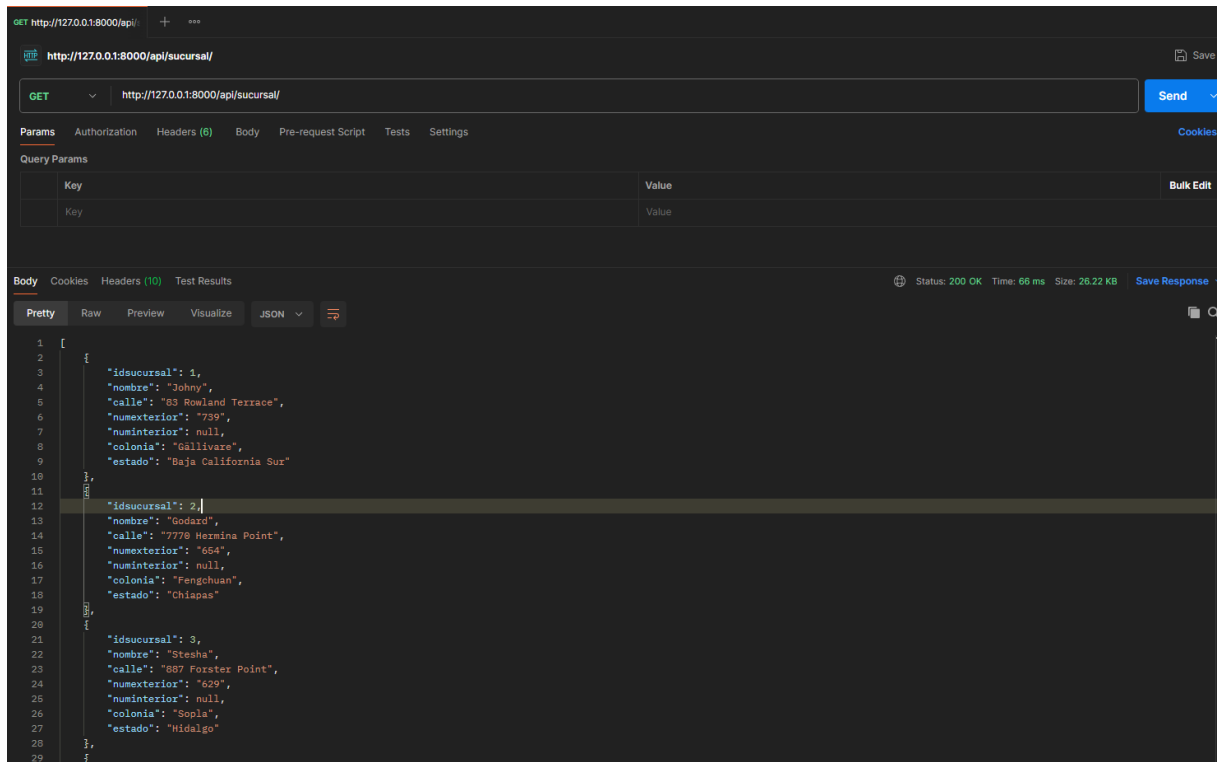
Esta operación elimina una sucursal mediante su identificador.

```
1 DELETE FROM sucursal  
2 WHERE idsucursal = %s;
```

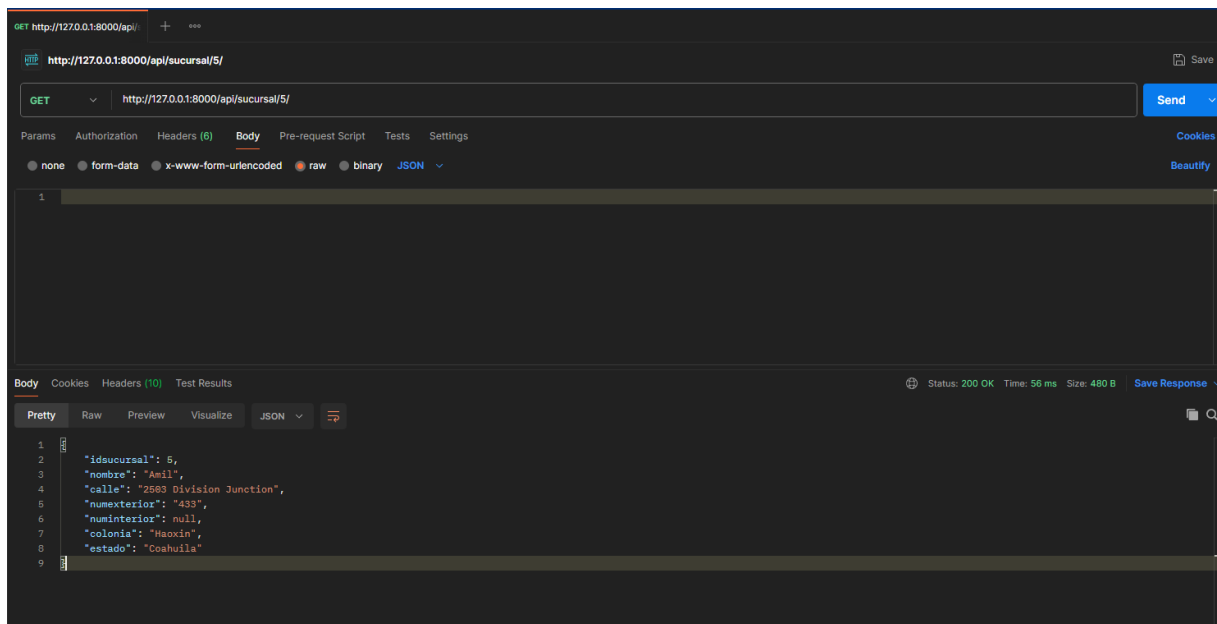
La query elimina únicamente la sucursal cuyo `idsucursal` coincida con el valor que recibió.

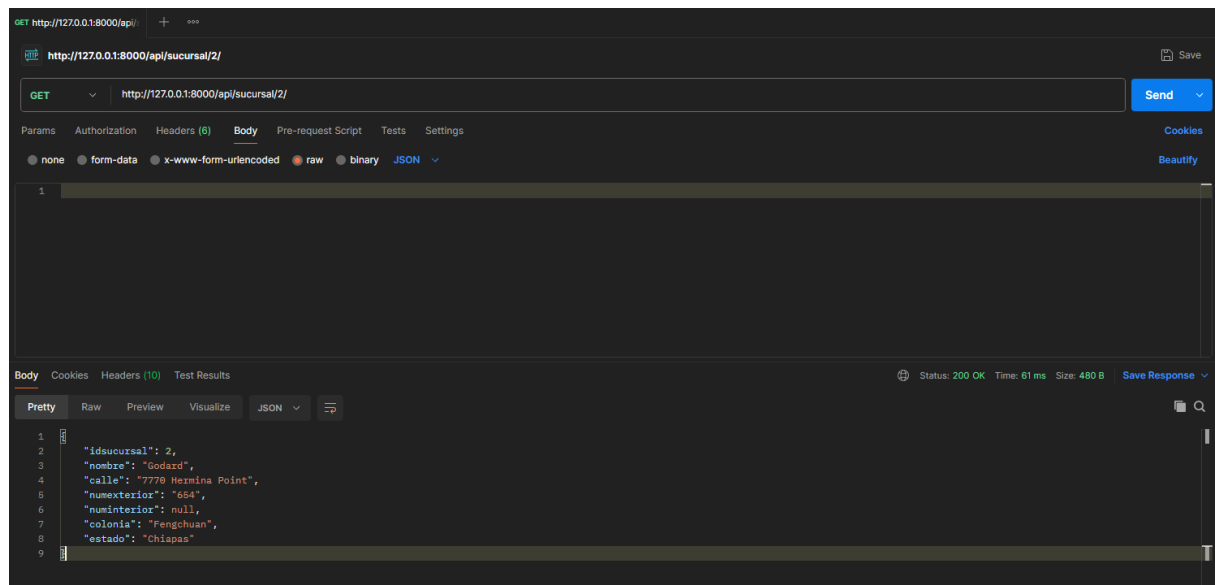
4. Evidencias de POSTMAN

- `get_all_clientes()`

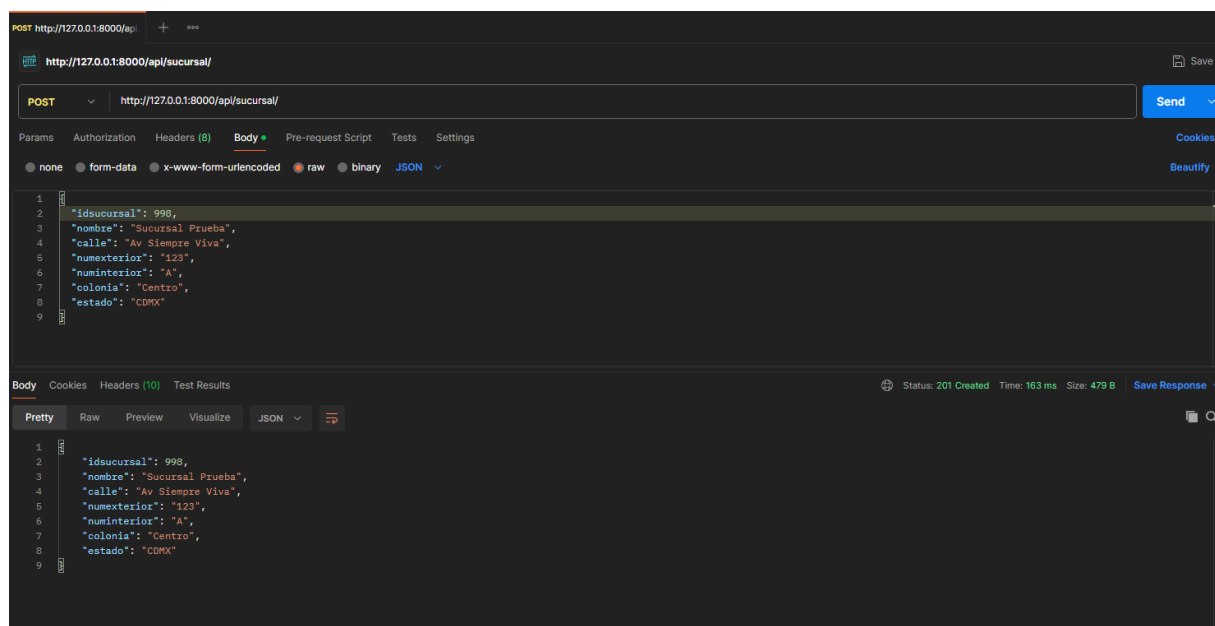


- `get_cliente_by_id(id_cliente)`

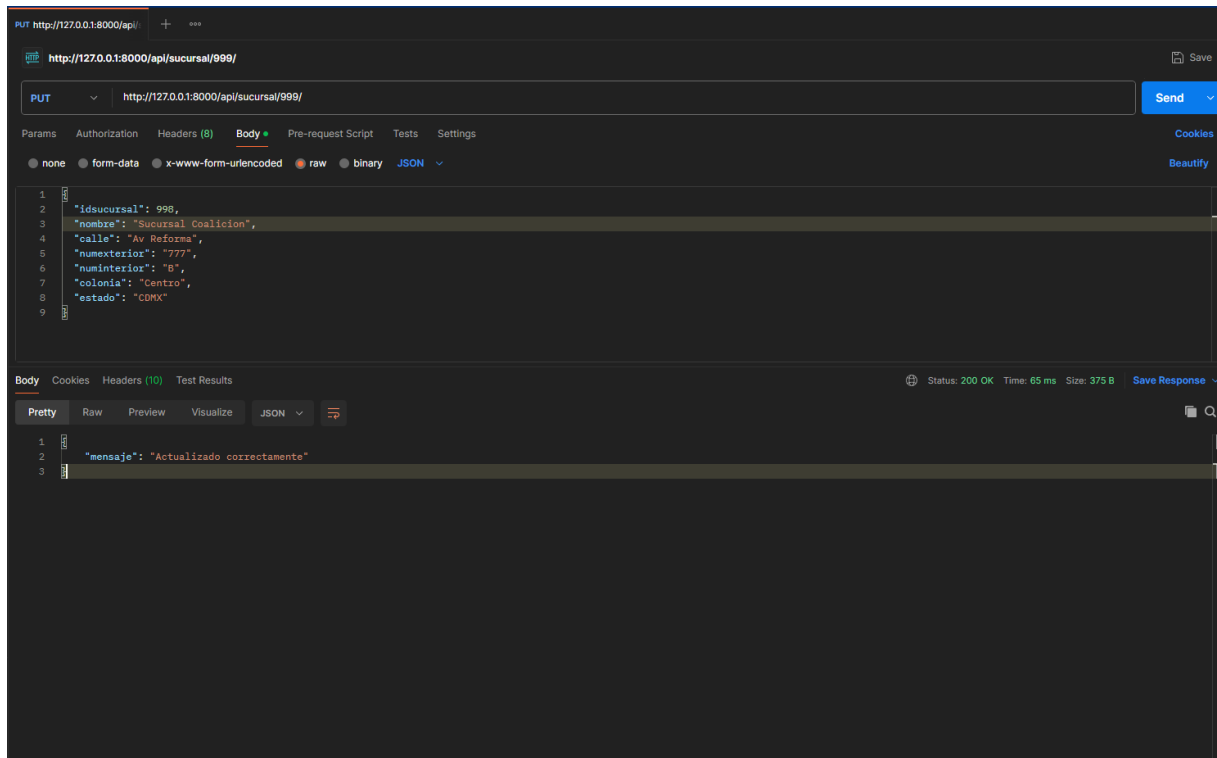




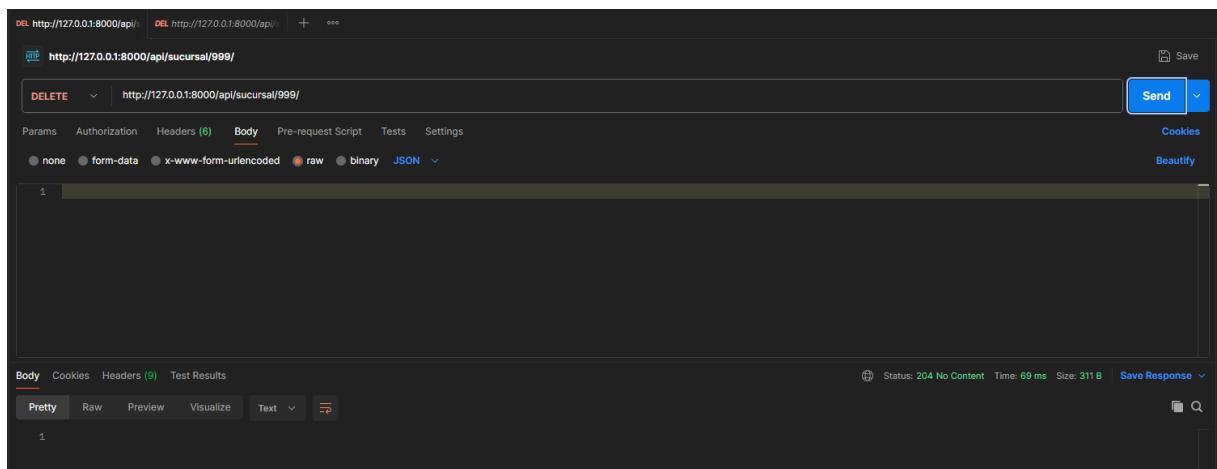
- `create_cliente(data)`

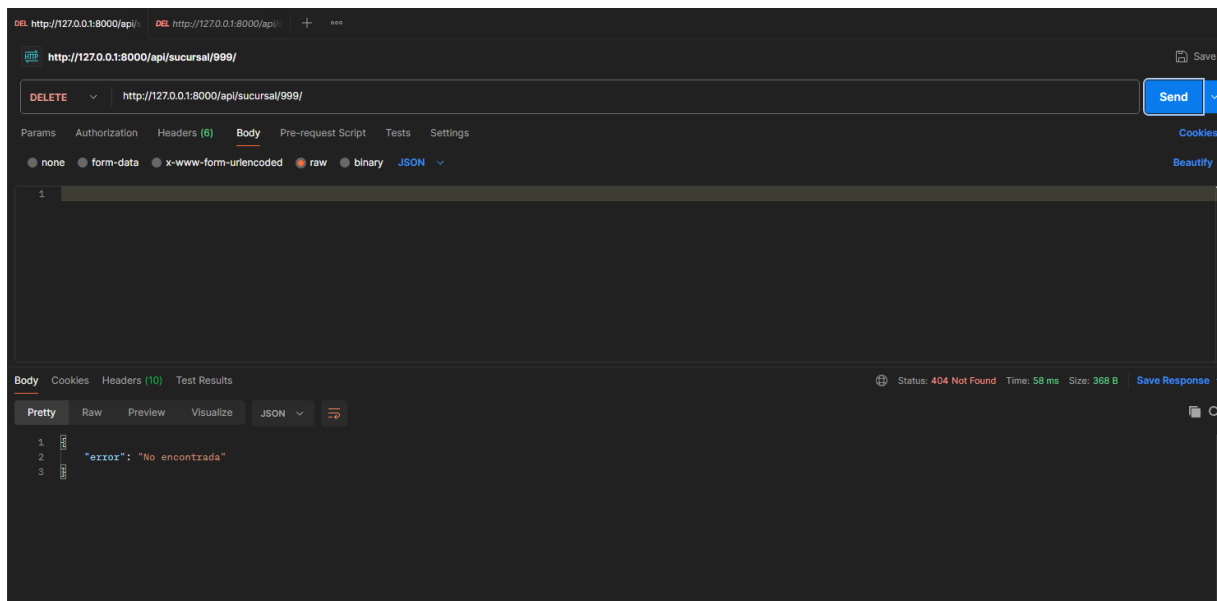


- `update_cliente(id_cliente, data)`



- delete_cliente(id_cliente)





5. Explicación de la aplicación

La aplicación desarrollada en esta práctica es una API REST conectada a una base de datos en **PostgreSQL**. Para su desarrollo se utilizó Django REST Framework como herramienta principal para crear los endpoints de la aplicación, mientras que la comunicación directa con la base de datos se realizó mediante Psycopg2. El objetivo principal de la aplicación fue permitir la ejecución de operaciones CRUD sobre dos tablas seleccionadas de la base de datos: **CLIENTE** y **SUCURSAL**.

La aplicación se organiza en diferentes partes, cada una encargada de una parte específica del funcionamiento. Primero, tenemos el archivo `manage.py`, que permite ejecutar el servidor de Django y administrar el proyecto, también se encuentra la carpeta principal de configuración del proyecto, donde están archivos como `settings.py` y `urls.py`. En estos archivos se definen aspectos importantes de la aplicación, como las aplicaciones instaladas, la configuración general del proyecto y las rutas principales que permiten acceder a los *endpoints*.

La conexión con la base de datos se centraliza en el archivo `db.py`. Este archivo contiene la función encargada de abrir la conexión con PostgreSQL utilizando Psycopg2. Con esta separación los demás archivos del proyecto no necesitan repetir la configuración de conexión cada vez que realizan una consulta. Lo que hacen es que llaman a esta función para conectarse a la base de datos y ejecutar las operaciones necesarias.

Para trabajar con la tabla **CLIENTE**, el proyecto cuenta con un módulo llamado `cliente`. Dentro de este módulo se encuentran principalmente los archivos `queries.py`, `views.py` y `urls.py`.

El archivo `queries.py` contiene las consultas SQL necesarias para realizar las operaciones CRUD sobre la tabla `CLIENTE`. Ahí se definen funciones para consultar todos los clientes, consultar un cliente por su identificador, insertar un nuevo cliente, actualizar los datos de un cliente existente y eliminar un cliente de la base de datos.

El archivo `views.py` del módulo `cliente` funciona como intermediario entre las peticiones HTTP y las consultas SQL. Cuando el usuario realiza una petición desde Postman o desde otra herramienta, la vista correspondiente recibe la solicitud, obtiene los datos necesarios y llama a la función adecuada dentro de `queries.py`. Por ejemplo, una petición `GET` permite consultar clientes, una petición `POST` permite insertar un nuevo cliente, una petición `PUT` permite actualizarlo y una petición `DELETE` permite eliminarlo.

De manera similar, para trabajar con la tabla `SUCURSAL`, el proyecto incluye un módulo llamado `sucursal`. Este módulo también contiene archivos `queries.py`, `views.py` y `urls.py`. En el archivo `queries.py` se encuentran las consultas SQL para obtener todas las sucursales, buscar una sucursal por su identificador, insertar una nueva sucursal, actualizar sus datos y eliminarla. Las vistas reciben las peticiones REST y llaman a estas funciones para ejecutar la operación correspondiente en la base de datos.

El flujo general de la aplicación inicia cuando el usuario realiza una petición a uno de los endpoints definidos. Django recibe la petición y la dirige al archivo de rutas correspondiente. Después, la vista procesa la solicitud y llama a una función del archivo `queries.py`. Esta función abre la conexión con la base de datos por medio de `db.py`, ejecuta la consulta SQL necesaria y regresa el resultado a la vista. Finalmente, la vista responde al usuario en formato JSON, mostrando la información consultada o un mensaje relacionado con la operación realizada.

Además, el proyecto incluye una interfaz sencilla en el archivo `farmacia_ui.html`. Esta interfaz permite interactuar de forma más visual con las operaciones CRUD, aunque las pruebas principales de la aplicación se realizaron mediante Postman. De esta forma, la aplicación cumple con el objetivo de conectar una base de datos PostgreSQL con una aplicación web, permitiendo manipular la información de las tablas `CLIENTE` y `SUCURSAL` usando endpoints REST.

6. Conclusión

En esta práctica se logró desarrollar una aplicación REST conectada a una base de datos en PostgreSQL, utilizando Django REST Framework y Psycopg2. A partir de esta conexión fue posible implementar operaciones CRUD sobre las tablas `CLIENTE` y `SUCURSAL`, permitiendo insertar, consultar, actualizar y eliminar registros directamente desde la aplicación.

El desarrollo de esta práctica permitió comprender mejor cómo una aplicación puede comunicarse con una base de datos sin manipularla directamente desde PostgreSQL. En lugar de ejecutar las consultas manualmente, se crearon endpoints que reciben peticiones HTTP y las traducen en operaciones SQL. Esto ayuda a entender la relación entre una base de datos, una aplicación web y una API REST.

También se reforzó el uso de consultas SQL como `SELECT`, `INSERT`, `UPDATE` y `DELETE`, aplicadas ahora dentro de un proyecto más funcional. También, las pruebas realizadas en Postman permitieron comprobar que los endpoints respondían correctamente y que las operaciones modificaban o recuperaban la información esperada de la base de datos.

En conclusión, la práctica fue útil para integrar varios temas vistos durante las prácticas, como el diseño de la base de datos, el uso de DDL y DML, la conexión con PostgreSQL y la implementación de servicios REST. Aunque la base de datos completa del proyecto contiene muchas más tablas, trabajar con **CLIENTE** y **SUCURSAL** permitió demostrar de forma clara el funcionamiento básico de una aplicación conectada a una base de datos real.